Sanskar's Dotfiles

A Stability-First NixOS Configuration

Complete system documentation — from bootloader to shell prompt

NixOS **25.11**	Kernel **6.12**	systemd-boot
Home Manager	KDE Plasma 6	NVIDIA PRIME

Sanskar Balpande
github.com/sanskar-0day/dotfiles

March 2026 · Generated from live configuration

Contents

1. Philosophy & Design 4
   1.1. Core Principles 4
   1.2. Architecture at a Glance 4
   1.3. Module Dependency Graph 5
2. Flake Structure 6
   2.1. Inputs 6
   2.2. Outputs 6
3. System Modules 7
   3.1. Boot & Kernel (`modules/boot.nix`) 7
   3.2. GPU Architecture (`modules/nvidia.nix`) 8
   3.3. Desktop Environment (`modules/desktop.nix`) 9
   3.4. Gaming (`modules/gaming.nix`) 10
   3.5. AI Infrastructure (`modules/ai.nix`) 11
   3.6. Virtualization (`modules/virtualization.nix`) 11
4. Host Configuration 13
   4.1. Networking 13
   4.2. Bluetooth 13
   4.3. Services 13
   4.4. Security 14
   4.5. Nix Daemon Tuning 14
5. Home Manager Configuration 15
   5.1. Shell (`shell.nix` + `zshrc`) 15
   5.2. Starship Prompt 16
   5.3. Git (`git.nix`) 16
   5.4. CLI Toolbelt (`tools.nix`) 16
   5.5. Neovim (`nvim.nix`) 17
   5.6. Plasma Configuration (`plasma.nix`) 17
   5.7. User Packages 17
6. Development Environments 18
   6.1. Philosophy 18
   6.2. Data Science Shell (`devshells/ds.nix`) 18
   6.3. Web Shell (`devshells/web.nix`) 18
   6.4. Dev Shell Access 18
7. Keyboard Layout (Kanata) 19
   7.1. Home-Row Modifiers 19
   7.2. Layers 19
8. Documentation Pipeline 21
9. Performance Tuning Summary 22
10. Bootstrapping & Maintenance 23

10.1. Fresh Install 23
10.2. Housekeeping 23
11. Troubleshooting 24

Philosophy & Design

I started using NixOS because I was frustrated with the fragility of traditional Linux setups. Every few months, something would break after an update — a driver mismatch, a config overwritten by a package manager, a library conflict that cascaded into three hours of debugging. NixOS eliminates that entire class of problems by making the system declarative and atomic.

This configuration has evolved over the past year from a single `configuration.nix` into a modular, flake-based setup. Every decision documented here was the result of actually hitting a problem and solving it — not cargo-culting from someone else's dotfiles.

Core Principles

Stability Over Novelty

The kernel is pinned to 6.12. NVIDIA uses the production driver branch. Unstable packages are only used where the stable version is missing or broken (AI tools, latest editors). I do not track `nixos-unstable` for the base system.

Reproducibility

Every aspect of the system is defined in this repository. A fresh NixOS install can be configured to a byte-identical state with `setup.sh` and a single rebuild. No manual steps, no "remember to also install X."

Minimal Surface Area

SSH is off. Avahi is off. Printing, Geoclue2, PackageKit — all disabled. The only network-facing services are the firewall (with KDE Connect ports) and Cloudflare WARP. Fewer running services means fewer things that can break or be exploited.

Fast Boot

systemd initrd for parallel module loading. Plymouth for visual feedback. Every service timeout set to 5 seconds. ModemManager disabled entirely. The goal is a usable desktop within seconds of POST.

Architecture at a Glance

The configuration follows a strict two-layer model:

System Layer (NixOS)

Kernel, bootloader, display manager, hardware drivers, system services, global packages, firewall. Defined across `hosts/` and `modules/`. Requires

User Layer (Home Manager)

Shell configuration, git, CLI tools, Neovim, Plasma settings, user packages, XDG directories. Defined in `home/sanskar/`. Rebuilds without root. Changes are instant — no reboot needed.

`sudo` to rebuild. Changes take effect on next boot (or live with `switch`).

Design Decision

Home Manager is injected as a NixOS module (not run standalone). This means `sudo nixos-rebuild switch` rebuilds *both* system and user configuration atomically. However, I also expose `homeConfigurations.sanskar` for quick user-only rebuilds via `hms` when I'm iterating on shell aliases or Plasma settings.

Module Dependency Graph

```
flake.nix
├── hosts/nixos/default.nix (main host)
│   ├── modules/boot.nix          ← systemd-boot, kernel, sysctl, plymouth
│   ├── modules/nvidia.nix        ← PRIME offload, Vulkan, VAAPI
│   ├── modules/desktop.nix       ← Plasma 6, PipeWire, portals
│   ├── modules/gaming.nix        ← Steam, GameMode, Gamescope
│   ├── modules/ai.nix            ← LM Studio NVIDIA wrapper
│   ├── modules/virtualization.nix ← Docker, QEMU/KVM
│   └── modules/typst.nix         ← Typst toolchain
│
├── home/sanskar/default.nix (user config)
│   ├── shell.nix     ← Zsh + Starship prompt
│   ├── git.nix       ← Git + Delta + GitHub CLI
│   ├── tools.nix     ← 50+ CLI tools
│   ├── nvim.nix      ← Neovim/LazyVim + 12 LSPs
│   ├── dev.nix       ← Formatters, runtimes, diagnostics
│   └── plasma.nix    ← Declarative KDE config
│
├── devshells/        ← Per-project dev environments
└── docs/builder.nix  ← Auto-generated PDF docs
```

Each module is self-contained. Removing an import line removes the entire feature with no dangling references. This composability is one of the strongest aspects of NixOS's module system.

Flake Structure

The flake is the top-level entry point. It pins all external inputs and wires them together.

Inputs

Input	Channel	Role
`nixpkgs`	25.11	System packages, kernel, drivers — the stable base
`nixpkgs-unstable`	unstable	Bleeding-edge packages (LM Studio, Ghostty, Zed, Cursor, Antigravity)
`home-manager`	25.11	User-space configuration management, follows nixpkgs
`nix-index-database`	rolling	Pre-built package index for `comma` (run without install)
`plasma-manager`	rolling	Declarative KDE Plasma settings via Home Manager

Design Decision

Both `home-manager` and `nix-index-database` use `inputs.nixpkgs.follows` to ensure they use the same `nixpkgs` revision as the system. This prevents diamond dependency issues where two inputs pull different versions of the same library.

Outputs

The flake exposes five outputs:

1. **`nixosConfigurations.nixos`** — The full system build. This is what `nixos-rebuild` targets.
2. **`homeConfigurations.sanskar`** — Standalone Home Manager for quick user rebuilds without sudo.
3. **`packages.x86_64-linux.docs`** — A Typst PDF auto-generated from the live configuration (via `docs/builder.nix`).
4. **`devShells.x86_64-linux.{ds, web}`** — Reproducible per-project development environments.
5. **`checks.x86_64-linux.nixos-config`** — CI-compatible build check that verifies the config evaluates cleanly.

The `checks` output is especially useful — running `nix flake check` before deploying catches evaluation errors, missing packages, and type mismatches without actually building the system.

System Modules

Boot & Kernel (`modules/boot.nix`)

This module controls the entire boot pipeline: bootloader, initrd, kernel parameters, plymouth splash, and systemd service timeouts.

Bootloader

systemd-boot is used over GRUB. It's faster (no config file generation), simpler, and integrates natively with systemd. Key settings:

- Editor disabled (`editor = false`) — prevents anyone from modifying kernel parameters at the boot menu, which is a security concern on shared machines
- Configuration limit of 10 generations — keeps the boot partition clean
- Timeout set to 0 — boots the default entry immediately. Hold Space during POST to access the menu when needed.

Kernel Parameters

```
quiet loglevel=3 splash fastboot noresume
nvidia-drm.modeset=1
nvidia.NVreg_PreserveVideoMemoryAllocations=1
mitigations=off
nowatchdog
amdgpu.dcfeaturemask=0x8
amdgpu.ppfeaturemask=0xffffffff
```

Each of these is a deliberate choice:

- `nvidia-drm.modeset=1` — Required for PRIME offload to function at all on modern NVIDIA drivers
- `nvidia.NVreg_PreserveVideoMemoryAllocations=1` — Prevents screen corruption after suspend/resume
- `mitigations=off` — Disables Spectre/Meltdown CPU mitigations for approximately 5–10% performance gain. This is an acceptable trade-off on a personal laptop that doesn't run untrusted code.
- `nowatchdog` — Disables the hardware watchdog timer, eliminating periodic interrupts that can cause micro-stutters
- `amdgpu.dcfeaturemask=0x8` — Enables the DC FP16 filter on the AMD iGPU, producing smoother frame output for window compositing
- `amdgpu.ppfeaturemask=0xffffffff` — Unlocks all power profile features for GPU tuning via CoreCtrl

Initrd

systemd initrd is enabled for parallelized module loading, which measurably reduces boot time compared to the default BusyBox initrd. Compression uses `zstd -6` instead of the default `-19` — this is 10× faster to *build* with identical decompression speed at boot.

Performance Sysctls

Parameter	Value	Rationale
`net.core.default_qdisc`	`fq`	Fair Queueing — works with BBR
`net.ipv4.tcp_congestion_control`	`bbr`	Google's congestion control — higher throughput
`vm.swappiness`	`10`	Strongly prefer RAM over swap
`vm.dirty_ratio`	`10`	Flush dirty pages to SSD sooner
`vm.dirty_background_ratio`	`5`	Start background writeback earlier
`vm.vfs_cache_pressure`	`50`	Keep filesystem metadata in cache longer
`kernel.nmi_watchdog`	`0`	Disable NMI watchdog (saves power)
`kernel.unprivileged_userns_clone`	`1`	Required for rootless containers

Service Timeouts

Every service timeout is set to 5 seconds:

```
systemd.settings.Manager = {
  DefaultTimeoutStartSec = "5s";
  DefaultTimeoutStopSec = "5s";
};
```

Individual slow services (flatpak-system-helper, NetworkManager, bluetooth, pipewire, sddm) are also clamped to 5 seconds. The reasoning: if a service can't start in 5 seconds, something is fundamentally wrong, and waiting 90 seconds (the default) just delays the inevitable.

GPU Architecture (`modules/nvidia.nix`)

PRIME Offload

This is an AMD+NVIDIA hybrid laptop. The AMD Radeon iGPU handles all desktop rendering (compositing, browser, file manager). The NVIDIA dGPU is powered down by default and only wakes when explicitly invoked via PRIME Offload.

Context	GPU	Method
Desktop	AMD Radeon	Default render — low power, silent fans
Games	NVIDIA	`nvidia-offload` wrapper (env vars + gamemoderun)
LM Studio	NVIDIA	`lmstudio-nvidia` wrapper (CUDA inference)
Video decode	NVIDIA	Hardware VAAPI via `nvidia-vaapi-driver`

NVIDIA Driver Settings

- **Production branch** — the most tested, stable driver

- **Open kernel module** — safe for Turing+ (RTX 20xx and newer)
- **Fine-grained power management** — the dGPU powers down completely when idle, which dramatically improves battery life
- **Modesetting enabled** — required for PRIME offload

The `nvidia-offload` Script

This isn't just `__NV_PRIME_RENDER_OFFLOAD=1`. The full script sets:

```
export __NV_PRIME_RENDER_OFFLOAD=1
export __NV_PRIME_RENDER_OFFLOAD_DESTINATION=NVIDIA
export __GLX_VENDOR_LIBRARY_NAME=nvidia
export __VK_LAYER_NV_optimus=NVIDIA_only
export __GL_THREADED_OPTIMIZATIONS=1     # Multithreaded OpenGL
export __GL_SHADER_DISK_CACHE=1          # Persistent shader cache
export __GL_SHADER_DISK_CACHE_SKIP_CLEANUP=1
export __GL_YIELD=USLEEP                 # Prevent CPU busy-spin
export __GL_MaxFramesAllowed=1           # Prevent frame queue overflow
exec gamemoderun "$@"                    # Maximum GPU clocks
```

Design Decision

`__GL_YIELD=USLEEP` is critical. Without it, the NVIDIA driver can enter a tight CPU polling loop at 100% load, causing system-wide stuttering. `__GL_MaxFramesAllowed=1` prevents the render queue from growing unbounded, which can crash the driver under heavy load.

Graphics Stack

The `hardware.graphics` configuration provides a complete Vulkan + video decode stack:

64-bit packages:

- `nvidia-vaapi-driver` — HW video decode
- `rocmPackages.clr.icd` — AMD OpenCL
- `vulkan-loader` + validation layers
- `libva` + VDPAU bridge

32-bit packages:

- `vulkan-loader` (pkgsi686Linux)
- Required for 32-bit games via Wine/Proton

Desktop Environment (`modules/desktop.nix`)

KDE Plasma 6 on X11

Caveat

Wayland support for NVIDIA has improved significantly, but Plasma 6 + NVIDIA + Wayland still has issues with XWayland scaling, screen flicker on wake, and SDDM crashes. X11 is the pragmatic choice for a stability-first configuration. SDDM Wayland is explicitly disabled.

Environment Variables

```
environment.sessionVariables = {
  KWIN_DRM_USE_MODIFIERS = "0";
  KWIN_TRIPLE_BUFFER = "1";
  KWIN_X11_NO_SYNC_TO_VBLANK = "1";
  KWIN_X11_FORCE_SOFTWARE_VSYNC = "1";
  QT_LOGGING_RULES = "*.debug=false;qt.qpa.wayland=false";
  ELECTRON_OZONE_PLATFORM_HINT = "x11";
};
```

`KWIN_TRIPLE_BUFFER` is the single most impactful setting for X11 smoothness on NVIDIA. Without it, KWin stutters visibly during window dragging and animations.

Bloat Removal

These packages are excluded from the Plasma install:

- `khelpcenter` — offline help (browser is enough)
- `elisa` — music player (VLC is better)
- `discover` — GUI package manager (Nix replaces this)
- `drkonqi` — crash reporter daemon
- `oxygen` — legacy Qt5 theme
- `kate` — text editor (Neovim is primary)
- `krdp` — remote desktop

Additionally, **Baloo** (file indexer) and **Akonadi** (PIM service) are neutralized by replacing their systemd services with `/bin/true`. Baloo alone can consume 300+ MB of RAM and cause constant disk thrashing.

Audio (PipeWire)

PipeWire is configured with a 64-quantum clock, giving 1.3ms latency at 48kHz:

```
"default.clock.quantum" = 64;    # 1.3ms (default 1024 = 21ms)
"default.clock.min-quantum" = 32;
"default.clock.max-quantum" = 512;
```

This matters for gaming (reduced audio lag) and for monitoring local audio. WirePlumber handles Bluetooth audio profile switching (A2DP ↔ HFP).

Gaming (`modules/gaming.nix`)

Stack

Component	Purpose
Steam	Native Linux + Proton compatibility layer
Proton-GE	Community Proton fork with extra game fixes
Gamescope	Micro-compositor for better frame pacing and FSR upscaling
GameMode	CPU governor + process priority + GPU clocks optimization
MangoHud	In-game FPS/thermal/frame-time overlay
Lutris	Game launcher for non-Steam games

Component	Purpose
Bottles	Wine prefix manager
Heroic	Epic Games / GOG launcher

Memory & Process Limits

Large games (especially compressed repacks) need special kernel settings:

- `vm.max_map_count = 2147483642` — prevents "out of memory" errors in Wine even when RAM is available (Wine maps many small memory regions)
- `fs.file-max = 524288` — higher open file limit
- Unlimited stack size — prevents segfaults during game decompression
- `nofile` limit raised to 524288 — some games open thousands of asset files simultaneously

Gaming Toggle (Kanata)

Home-row modifiers interfere with WASD movement in games. Two aliases solve this cleanly:

- `game-on` — stops the Kanata systemd service, all keys behave normally
- `game-off` — restarts Kanata, home-row modifiers are restored

These are system-level operations (require doas/sudo).

AI Infrastructure (`modules/ai.nix`)

LM Studio is the primary local inference tool. The `lmstudio-nvidia` wrapper forces CUDA execution on the dGPU with all the same performance flags as `nvidia-offload`, plus `CUDA_CACHE_DISABLE=0` to persist CUDA kernel caches.

The model directory is managed via a Home Manager activation:

```
home.activation.linkLmStudioModels = lib.hm.dag.entryAfter ["writeBoundary"] ''
  mkdir -p ~/.cache/lm-studio
  ln -sfn ~/models ~/.cache/lm-studio/models
'';
```

This means models stored in `~/models` are automatically available in LM Studio without copying or manual configuration.

Additionally, five AI CLI tools are pulled from the unstable channel: `codex`, `gemini-cli`, `qwen-code`, `opencode`, and `antigravity`.

Virtualization (`modules/virtualization.nix`)

Docker

Docker is **socket-activated** — it doesn't start on boot. The first `docker` command triggers the daemon. This saves boot time and RAM when Docker isn't needed.

Logging uses the `local` driver with 10MB per container and 3 rotated files, preventing unbounded log growth that can fill an SSD over time.

QEMU/KVM

libvirtd is configured with:

- `qemu_kvm` package for hardware-accelerated virtualization
- TPM emulation (swtpm) — required for Windows 11 VMs
- Virt-manager GUI for VM management
- VirtIO driver ISO for Windows guest performance
- Spice GTK for USB passthrough

Host Configuration

The main host config (`hosts/nixos/default.nix`) ties everything together and handles machine-specific settings that don't belong in reusable modules.

Networking

NetworkManager with WiFi power saving disabled (causes connection drops on the TP-Link adapter). The TCP stack is tuned with larger buffer sizes (16MB) and TCP Fast Open enabled.

DNS goes through `systemd-resolved` with DNS-over-TLS (opportunistic mode) and fallback servers from Cloudflare, Quad9, and Google. Cloudflare WARP is enabled as a system service.

The firewall allows only KDE Connect ports (1714–1764 TCP/UDP) and uses loose reverse path checking for VPN compatibility.

Bluetooth

Bluetooth is configured for maximum compatibility with consumer earbuds and Bluetooth mice:

- Powered off at boot (saves battery)
- `FastConnectable = true` — faster pairing
- `MultiProfile = "multiple"` — supports A2DP + HFP simultaneously
- Low minimum encryption key size (7) — some earbuds won't pair otherwise
- 15 reconnection attempts at 1-second intervals

The `bt-fix` and `bt-clean` aliases handle the most common Bluetooth issue: "device is visible but won't connect."

Services

Service	State	Rationale
openssh	off	No remote access needed
flatpak	on	Alternative package source
cloudflare-warp	on	VPN / DNS privacy
avahi	off	mDNS not needed
printing	off	No printer
geoclue2	off	Location services not needed
packagekit	off	Nix is the package manager
ModemManager	off	Dramatically speeds up boot
fstrim	on	Weekly SSD TRIM
irqbalance	on	Distribute IRQs across cores

Service	State	Rationale
ananicy	on	CachyOS rules for process priority
earlyoom	on	Kill at 5% free RAM

Security

- Both `sudo` and `doas` are enabled, with `sudo` aliased to `doas`
- `doas` configured with `keepEnv` and `persist` for the primary user
- Login fail delay set to 0 (was 4 seconds — unnecessary on a personal machine with full-disk encryption)
- Boot editor disabled in systemd-boot

Nix Daemon Tuning

The Nix daemon is aggressively throttled to prevent builds from making the desktop laggy:

```
nix.daemonCPUSchedPolicy = "idle";
nix.daemonIOSchedClass = "idle";
systemd.services.nix-daemon.serviceConfig = {
  CPUWeight = 20;
  CPUQuota = "60%";
  OOMScoreAdjust = 500;
};
```

Builds run at the lowest CPU and I/O priority. The daemon is limited to 60% CPU and gets killed first by OOM. This means that even a `nix build` of the entire system won't cause a stutter in KWin or drop audio.

Home Manager Configuration

Shell (`shell.nix` + `zshrc`)

The shell config is intentionally split:

- `shell.nix` — declarative Home Manager options (history size, plugins, options, aliases)
- `zshrc` — imperative configuration (keybindings, functions, integrations)

Key Features

100K history entries with deduplication, space-ignoring, and timestamp recording. History is shared across all terminal sessions in real-time.

Emacs keybindings in the shell (I use vim bindings in the editor, but emacs-mode in the terminal is more intuitive for line editing). Magic Ctrl-Z toggles between foreground and background — press Ctrl-Z with an empty buffer and it runs `fg`, otherwise it pushes the current input to a stack.

Shell.nix auto-activation — a `chpwd` hook detects `shell.nix` in the current directory and enters the nix-shell automatically. This is incredibly useful for coursework projects:

```
$ cd ~/projects/mad1-project
❄  shell.nix detected — entering nix-shell...
# Automatically inside the project's Nix environment
```

It tracks the active directory via `_NIXSHELL_DIR` so it won't re-enter if you're already inside, and cleans up when you exit.

Aliases

Over 50 aliases covering system maintenance, development, navigation, and gaming:

Alias	Command	Category
`nrs`	`nh os switch ~/dotfiles`	System rebuild
`hms`	`home-manager switch --flake ...`	User rebuild
`ds`	`nix develop ~/dotfiles#ds`	Enter DS devshell
`steam-nv`	`nvidia-offload steam`	Steam on NVIDIA
`game-on`	Stop Kanata service	Gaming mode
`bt-fix`	Reset BT + audio stack	Troubleshooting
`pdf`	`typst compile`	Document compilation
`dots`	`cd ~/dotfiles && $EDITOR .`	Quick edit

Custom Functions

- `dev()` — lists available devShells or enters one by name
- `ns()` / `nu()` — fuzzy search stable/unstable packages with fzf preview
- `ni()` — quick `nix shell` for one-off package use
- `extract()` — universal archive extraction
- `mkcd()` — create directory and cd into it
- `wifi-mt()` / `wifi-fix()` — toggle between internal MediaTek and external TP-Link WiFi adapters

Starship Prompt

Minimal prompt: `directory → git_branch → git_status → nix_shell → character`

The character symbol is the NixOS snowflake (❄) — green for success, red for error, purple for vi-command mode. The prompt is deliberately fast (`scan_timeout = 10`).

Git (`git.nix`)

Git with Delta for side-by-side syntax-highlighted diffs (Dracula theme). Rebase on pull. Auto-setup remote on push. GitHub CLI with the dashboard extension.

CLI Toolbelt (`tools.nix`)

Over 50 modern CLI tools are managed here, organized by category:

Search & Files

- `ripgrep` — fast grep
- `fd` — fast find
- `fzf` — fuzzy finder
- `eza` — modern ls
- `bat` — cat with syntax
- `yazi` — TUI file manager

Git & Version Control

- `lazygit` — TUI git client
- `delta` — beautiful diffs
- `git-extras` — extra commands
- `gh` — GitHub CLI + dashboard

System & Monitoring

- `btop` — system monitor
- `bandwhich` — network monitor
- `procs` — modern ps
- `dust` / `duf` / `dua` — disk tools
- `bottom` — alternative monitor

Nix-Specific

- `nh` — nix helper
- `nvd` — generation diff
- `nix-output-monitor` — pretty builds
- `nix-fast-build` — parallel builds
- `nurl` — fetch expression generator
- `comma` — run without install
- `any-nix-shell` — zsh in nix-shell
- `devenv` — dev environments

Each tool is configured through Home Manager programs when possible (fzf integration, zoxide, direnv, atuin settings) — not just installed.

Neovim (`nvim.nix`)

LazyVim-based configuration with Nix-managed dependencies. All LSP servers, formatters, and debug adapters are declared in Nix — not installed via Mason — ensuring they're always available and at consistent versions.

Category	Tools
LSP Servers	pyright, ruff, lua-language-server, nil (Nix), zls (Zig), nimlangserver, typescript-language-server, vscode-langservers-extracted, taplo (TOML), yaml-language-server, marksman (Markdown), sbcl (Lisp)
Formatters	black, stylua, nixfmt-rfc-style, typstyle, prettier, shfmt
Linters	shellcheck, ruff
Debug	debugpy (Python DAP)
AI Plugins	avante.nvim (Cursor-like), CodeCompanion, CopilotChat, NeoCodeium

Plasma Configuration (`plasma.nix`)

Declarative KDE Plasma settings via `plasma-manager`:

- Animation speed set to 0.2 (nearly instant)
- Blur and wobbly windows disabled for maximum performance
- Splash screen disabled
- Font stack: Inter (UI), JetBrains Mono Nerd Font (code)
- Consistent font sizes across all UI elements

User Packages

The user package list in `default.nix` includes browsers (Firefox), messaging (Telegram), media (VLC), AI tools (LM Studio, Antigravity), editors (Ghostty, Zed, Cursor), and a comprehensive Typst package collection (80 packages for presentations, CVs, diagrams, and academic writing).

Development Environments

Philosophy

Global developer tools are limited to language-agnostic essentials: `git`, `just`, `tokei`, `hyperfine`, formatters (nixfmt, shfmt, prettier), and light runtimes (`uv`, `bun`). Heavy, project-specific stacks are isolated in flake devShells.

This means the base system stays lean and fast. You don't pay the evaluation cost of a Python environment when you're doing web development.

Data Science Shell (`devshells/ds.nix`)

This is my IITM BS coursework environment. It includes everything needed for the Modern Application Development courses:

Python (30+ packages):
- Flask + SQLAlchemy + Migrate
- Flask-RESTful + Flask-Security
- Celery + Redis
- JWT + bcrypt + passlib
- marshmallow + Pydantic
- pytest + coverage + factory-boy
- black + ruff + mypy
- gunicorn (production)

System Tools:
- SQLite + litecli + sqldiff
- Redis server
- Node.js 22 (Vue frontend)
- httpie + jq + websocat
- just + watchexec + entr
- ripgrep + fd + bat
- git + gh

The shell hook sets up Flask defaults, Redis URLs, and convenience aliases (`flask-run`, `redis-start`, `db-shell`, `celery-worker`). Entering the shell prints a banner with version info and available commands.

Web Shell (`devshells/web.nix`)

Minimal stack for web freelancing: Node.js 22, Bun, Git, Just.

Dev Shell Access

From the shell:

```
$ dev            # List available shells
$ dev ds         # Enter Data Science shell
$ nix develop .#web  # or via nix directly
```

Keyboard Layout (Kanata)

Kanata runs as a systemd service, intercepting keystrokes at the OS level before they reach applications. The configuration lives at `hosts/nixos/kanata.kbd` and is processed with `process-unmapped-keys yes`, meaning only explicitly remapped keys are affected.

Home-Row Modifiers

The core innovation: the eight home-row keys produce their normal letter on tap and a modifier on hold (200ms threshold):

A	S	D	F	J	K	L	;
Super	NumSym	Ctrl	Shift	Shift	Ctrl	NumSym	Super

The symmetry is intentional — left pinky mirrors right pinky, left index mirrors right index. Modifier chords always use one hand per side, just like a full-size keyboard.

Design Decision

`S` and `L` activate the NumSym layer instead of Alt. Alt is available through the Right Alt key (which also serves as the NumSym layer trigger on hold). This means the most-used layers (navigation via CapsLock, symbols via S/L) are on the strongest fingers.

Layers

Three keys unlock additional layers:

CapsLock (hold) → Navigation:

- `H/J/K/L` = Arrow keys (vim-style)
- `U/I` = Home / End
- `O/P` = Page Up / Page Down
- `Z/X/C/V/B` = Undo / Cut / Copy / Paste / Redo
- Top row = F1–F12

Tab (hold) → System:

- `H/J/K/L` = Mouse movement (30px steps)
- `U/I` = Scroll up/down
- `C/V/B` = Mouse left/right/middle click
- Bottom row = Volume, mute, play/pause, prev/next track

Right Alt (hold) → NumSym:

- Top row = shifted symbols (`! @ # $ % ^ & * ( )`)

- Middle row = plain numbers (`1 2 3 4 5 6 7 8 9 0`)
- Bottom row = brackets, braces, math operators (`+ = \\ |`)

Documentation Pipeline

`nix build .#docs` generates a complete PDF reference from the live configuration. The builder (`docs/builder.nix`) reads every `.nix` file via `builtins.readFile` at Nix evaluation time and interpolates them into a Typst template.

This means the documentation can never go stale — it's generated from the same source that `nixos-rebuild` uses.

The output includes:
- Styled title page with version information
- Auto-generated table of contents
- Every module with syntax-highlighted code blocks
- Explanatory callouts for each section

Alias: `build-docs` compiles and copies the PDF to the docs directory.

Performance Tuning Summary

Area	Optimization	Impact
CPU	ananicy-cpp + CachyOS rules	KWin/Plasma stay responsive under load
Memory	EarlyOOM at 5% threshold	Prevents total system lockup
Memory	zram (zstd, 50%)	Effective RAM nearly doubled
I/O	NVMe: `none` scheduler	Drive handles its own queuing
I/O	SATA: `bfq` scheduler	Better interactive responsiveness
Network	TCP BBR + FQ qdisc	Higher throughput, lower latency
Network	TCP Fast Open	Fewer round-trips per connection
Nix	60% CPU cap, idle priority	Builds are invisible to desktop
Boot	systemd initrd, 5s timeouts	Fast, predictable startup
Audio	PipeWire 64-quantum	1.3ms latency (from 21ms default)
Fonts	Subpixel RGB, slight hinting	Sharp text on LCD panels
Desktop	Baloo + Akonadi disabled	300MB RAM saved, no disk thrashing
GPU	Fine-grained NVIDIA PM	dGPU powers down when idle

Bootstrapping & Maintenance

Fresh Install

```
bash <(curl -sL https://raw.githubusercontent.com/sanskar-0day/dotfiles/main/setup.sh)
```

The script handles pre-flight checks (not root, internet connectivity), enables flakes, installs git, clones the repo, and copies `hardware-configuration.nix`. Then:

```
sudo nixos-rebuild boot --flake ~/dotfiles#nixos
```

Reboot, run `hms`, authenticate with GitHub, generate SSH keys — and the system is identical to the original.

Housekeeping

- **Journal logs**: Capped at 1GB with 3-month retention
- **Core dumps**: Stored in journal, 500MB max per dump, compressed
- **Nix store**: Auto-optimized (hard-link deduplication)
- **Garbage collection**: `nh clean` keeps last 3 generations and everything from the past 4 days
- **Shader caches**: Pre-created directories prevent first-launch stutters in games

Troubleshooting

Slow boot

`systemd-analyze blame` — shows the slowest services. `systemd-analyze critical-chain` — shows the boot dependency chain. Common culprit: `NetworkManager-wait-online` (disabled in this config).

NVIDIA not detected

Run `nvidia-smi` to confirm the driver loaded. Check `lsmod | grep nouveau` returns nothing. Use `nvtop` for real-time GPU monitoring. If the driver version changed, `nrb` + reboot is safer than `nrs`.

Game runs on AMD instead of NVIDIA

The game must be launched via `nvidia-offload`. For Steam, set launch options to: `nvidia-offload gamemoderun %command%`. Verify with `nvidia-smi` while the game is running.

Bluetooth won't pair

Run `bt-fix` (resets rfkill + bluetooth daemon + PipeWire/WirePlumber). If that fails, run `bt-clean` (unpairs all devices and resets). Check `rfkill list` for soft-blocked adapters.

Audio crackling or dropouts

PipeWire may need a larger quantum for Bluetooth devices. Try: `pw-metadata -n settings 0 clock.force-quantum 1024`. If persistent, WirePlumber may need a restart: `systemctl --user restart wireplumber`.

Build fails with "collision" errors

Usually means two packages provide the same file. Check which packages conflict with `nix why-depends` and resolve by removing one or using `lib.mkForce` on the priority.

Sanskar Balpande

Built with NixOS 25.11 · Kernel 6.12 · Home Manager

github.com/sanskar-0day/dotfiles

Last updated March 2026